

Lab 7. Analysis of Monthly Airline Passenger Numbers with statsmodels

During this mini-project, we will work with the dataset `AirPassengers.csv`. It contains monthly totals of US airline passengers from 1949 to 1960.

This dataset can be represented as a time series. A time series is a sequence of observations recorded and ordered in time. Formally, it is defined as a collection of random variables

$$\{Y_t\}_{t \in T},$$

where Y_t is observed at regular time intervals (e.g., daily, monthly, yearly).

```
In [63]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Loading and plotting the data

Task 1. (Loading the Dataset)

Loading the Dataset `AirPassengers.csv`.

Instructions:

- Download the dataset `AirPassengers.csv`.
- Set the index as a datetime object using: `pd.to_datetime(passengers.index)`
- Print the first few rows of the dataset with `.head()`.

Remark: The function `pd.to_datetime` in pandas converts strings (like `"1949-01"`) into proper datetime objects.

```
In [2]: passengers=pd.read_csv("AirPassengers.csv", index_col="Month")
passengers.index=pd.to_datetime(passengers.index)
passengers.head()
```

Out[2]: **#Passengers**

Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Task 2. (Plotting the data)

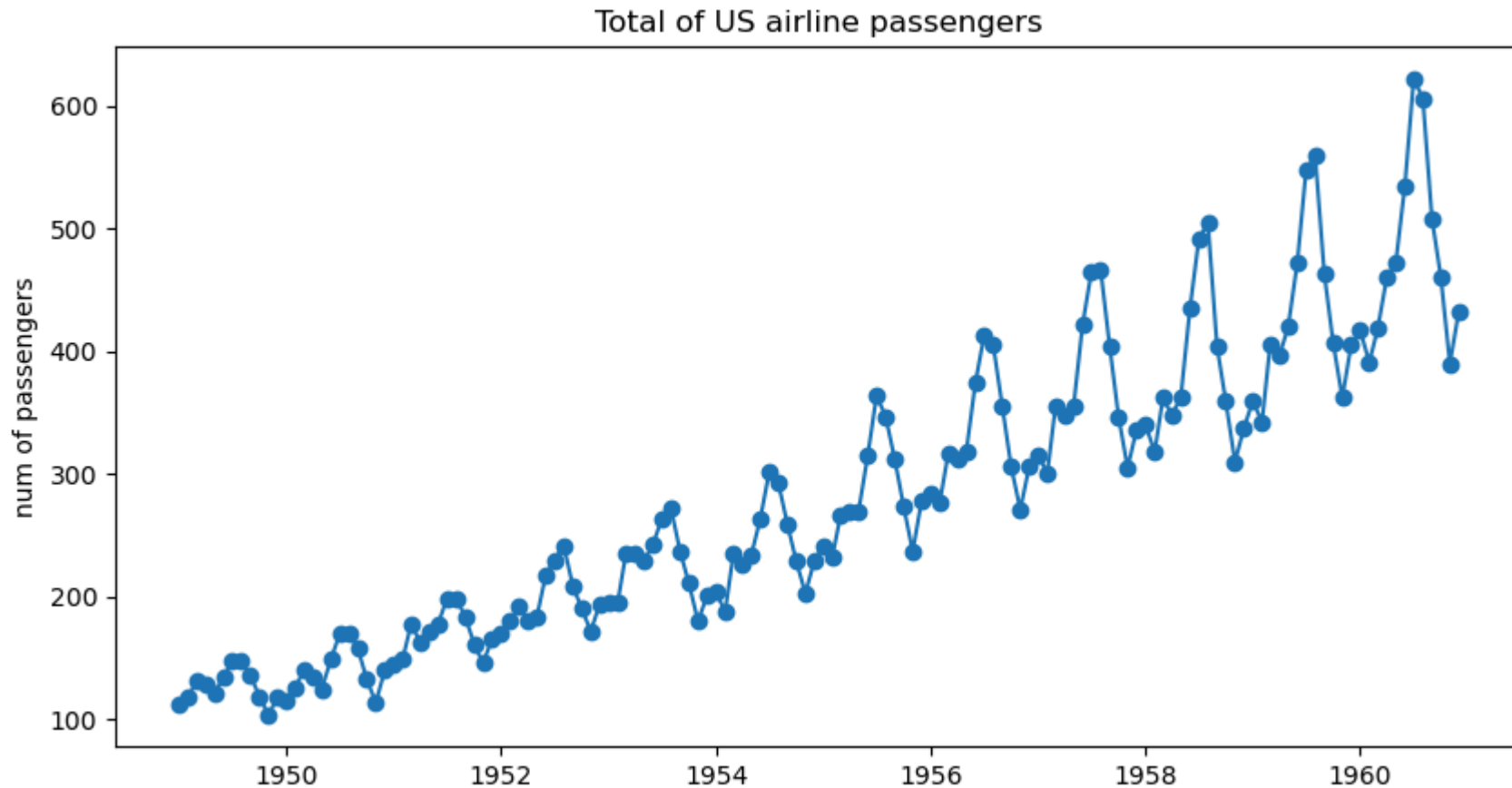
Plot the dataset using the pandas `.plot()` method.

Instructions:

- Set the figure size with `figsize=(10, 5)`.
- Add a descriptive `title` and `ylabel`.
- Use markers on the data points with `marker="o"`.

```
In [54]: plt.figure(figsize=(10,5))
plt.plot(passengers, marker="o")
plt.title("Total of US airline passengers")
```

```
plt.ylabel("num of passengers")  
plt.show()
```



Task 3. (Train–Test Split)

Before further analysis and prediction, we need to split the dataset into training and testing parts. This allows us to evaluate how well models perform on unseen data.

Instructions:

1. Check the total number of observations in the dataset.

2. Split the series into two arrays:

- `passengers_train` = first ~95% of the data (used for model training),
- `passengers_test` = remaining ~5% of the data (used for model evaluation).

```
In [4]: datalen=len(passengers)
tr_size=int(0.95*datalen)

passengers_train=passengers[:tr_size]
passengers_test=passengers[tr_size:]

print(f>Data size: {datalen}, train size: {len(passengers_train)}, test size: {len(passengers_test)}.)
```

Data size: 144, train size: 136, test size: 8.

Decomposition of Time Series

From the plot, we can identify the following characteristics:

- An upward trend over time.
- Clear seasonality with a 12-month period (because the data is monthly).
- Increasing variance, as fluctuations grow larger in later years.

The data can be represented using the following additive decomposition model:

$$Y_t = T_t + S_t + X_t, \quad t \in \mathbb{Z},$$

where:

- T_t – deterministic trend component,
- S_t – deterministic seasonal component,
- X_t – random (irregular) component.

Stationarity

In time series analysis, we often want to work with stationary data.

A time series $\{Y_t\}_{t \in \mathbb{Z}}$ is called *stationary* if its mean and autocovariance function are constant over time, i.e.

$$\mathbb{E}[Y_t] = \mu, \quad \text{Cov}(Y_t, Y_{t+\tau}) = \gamma(\tau),$$

where μ is a constant mean and $\gamma(\tau)$ depends only on the lag τ , not on the specific time t .

To obtain a stationary process from our data, we will remove the trend and the seasonal component from the data.

Task 4. (Removing the Trend)

A trend is the long-term increase or decrease in the data. For example, in the `AirPassengers` dataset we observe a steady upward movement over the years.

One way to remove the trend is to compute the first difference between consecutive observations:

$$Y'_t = Y_t - Y_{t-1} = \underbrace{(T_t - T_{t-1})}_{T'_t} + \underbrace{(S_t - S_{t-1})}_{S'_t} + \underbrace{(X_t - X_{t-1})}_{X'_t}.$$

Note that:

- If the trend is linear, differencing removes it completely.

$$T'_t = T_t - T_{t-1} = \beta_1,$$

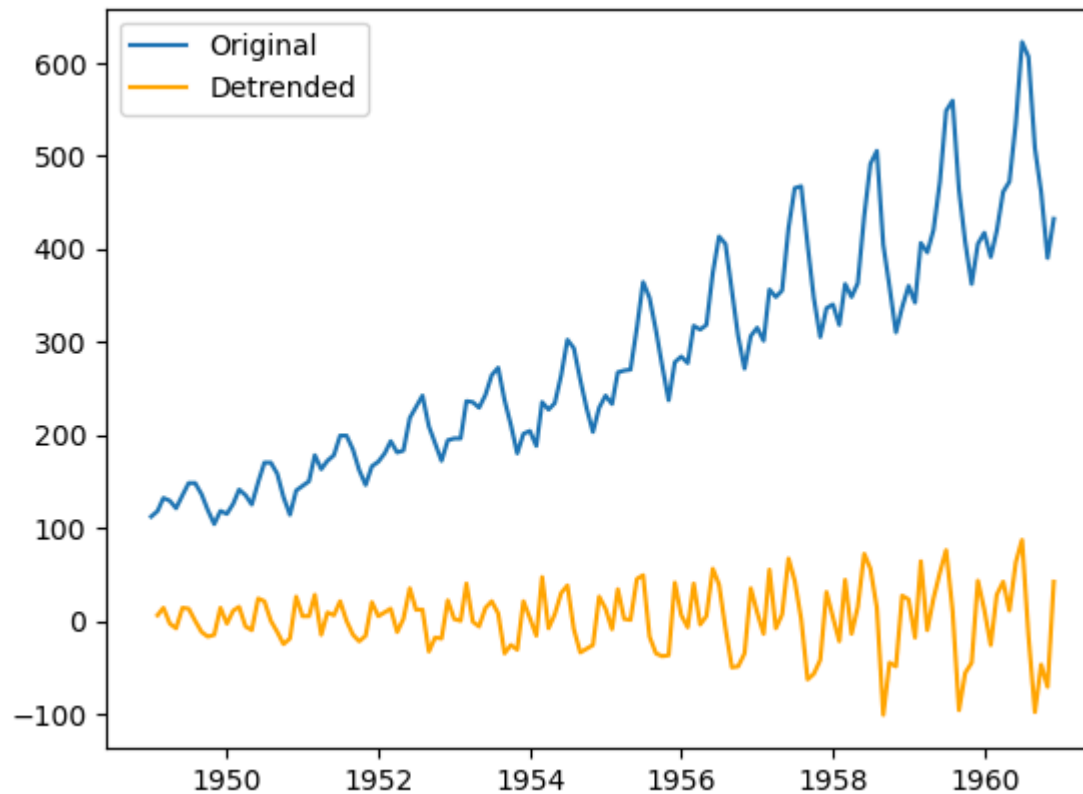
- If the trend is nonlinear, more rounds of differencing may be needed.

Instructions:

- Use `.diff()` in pandas to compute first differences.
- `.diff()` creates `NaN` in the first row – drop it with `.dropna()`.
- Store the result in `passengers_diff`.

- Plot the detrended series alongside the original data for comparison.

```
In [53]: passengers_diff=passengers.diff().dropna()
plt.plot(passengers, label="Original")
plt.plot(passengers_diff, label="Detrended", color="orange")
plt.legend()
plt.show()
```



Task 5. (Removing Seasonality)

After removing the trend, the next step is to remove the seasonal component.

A common way to do this is to compute the d -th difference, where d equals the seasonal period (for monthly data with yearly seasonality, $d = 12$):

$$Y_t'' = Y_t' - Y_{t-d}' = \underbrace{(T_t' - T_{t-d}')}_{T_t''} + \underbrace{(S_t' - S_{t-d}')}_{S_t''} + \underbrace{(X_t' - X_{t-d}')}_{X_t''}.$$

Instructions:

- Use `.diff(12)` on `passengers_diff`.
- Store the result in `passengers_diff_1_12`.
- Drop `NaN` values with `.dropna()`.
- Create two plots:
 - `passengers_diff_1_12` together with `passengers_diff`.
 - Only `passengers_diff_1_12` with a horizontal line at zero using `plt.hlines`.

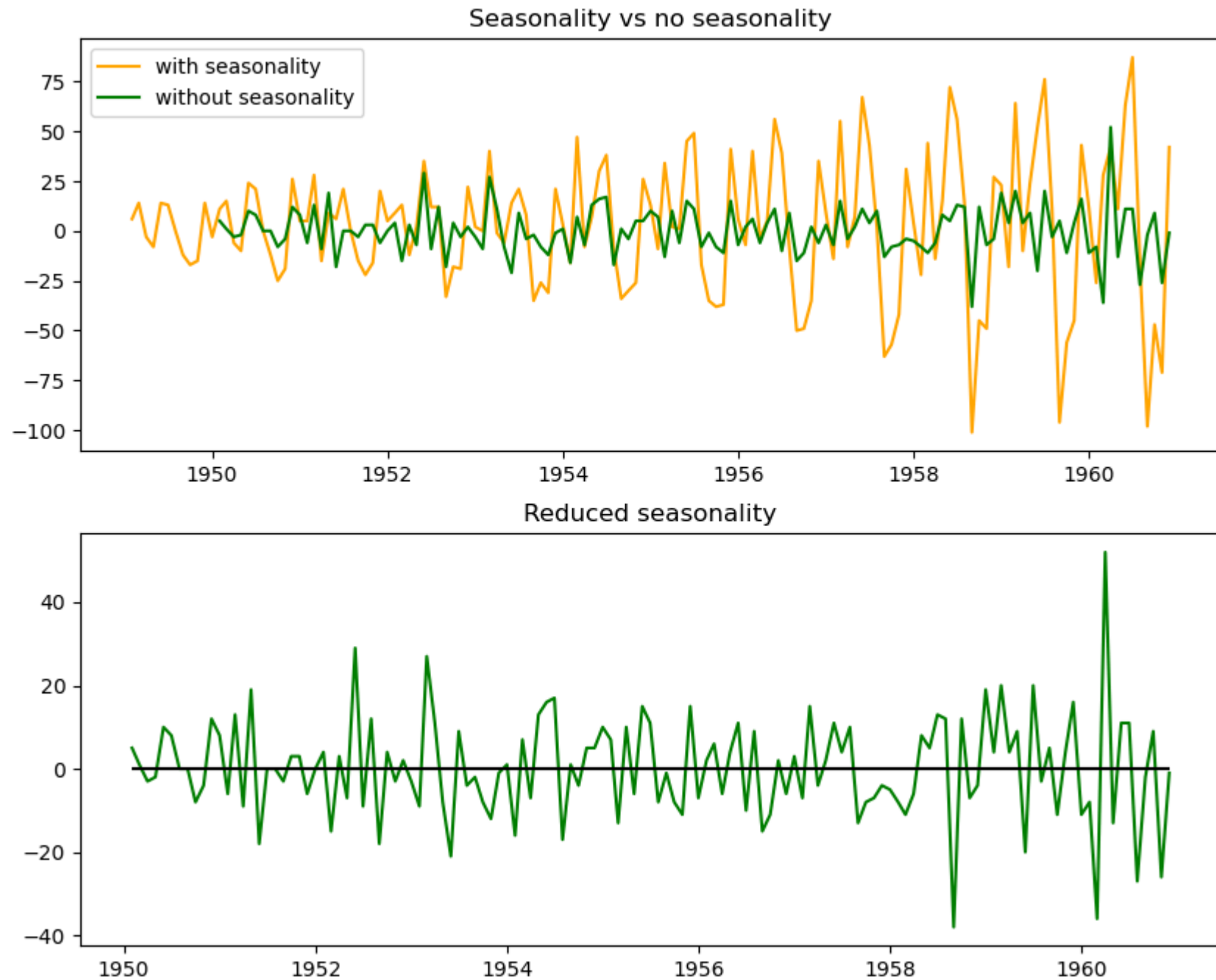
```
In [40]: passengers_diff_1_12=passengers_diff.diff(12).dropna()
passdifflen=len(passengers_diff_1_12)

fig, axes=plt.subplots(2, 1, figsize=(10, 8))

axes[0].plot(passengers_diff, label="with seasonality",color="orange")
axes[0].plot(passengers_diff_1_12, label="without seasonality", color="green")
axes[0].set_title("Seasonality vs no seasonality")
axes[0].legend()

axes[1].plot(passengers_diff_1_12, color="green")
axes[1].hlines(y=0, xmin=passengers_diff_1_12.index[0], xmax=passengers_diff_1_12.index[passdifflen-1], color="black")
axes[1].set_title("Reduced seasonality")

plt.show()
```



ARMA(p, q) model

After applying the necessary transformations, the series now appears to be stationary.
With stationary data, we can apply various statistical tools designed for such processes.

In particular, we can model the stationary series using popular parametric models such as AR, MA, or ARMA.

An autoregressive moving average model ARMA(p, q) is defined as:

$$X_t = c + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + Z_t + \theta_1 Z_{t-1} + \cdots + \theta_q Z_{t-q},$$

where $Z_t \sim \text{WN}(0, \sigma^2)$ is white noise (iid data).

Interpretations:

1. Autoregressive (AR) part: Captures persistence or dependence on the past values of the series.
 2. Moving Average (MA) part: Captures dependence on past random fluctuations.
 3. ARMA combination: Captures both persistence and random fluctuations.
-

Stationarity:

A stationary solution exists if and only if the autoregressive polynomial

$$\phi(z) = 1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_p z^p \neq 0 \quad \text{for all } |z| = 1,$$

i.e., the roots of $\phi(z)$ must lie outside the unit circle.

Autocorrelation and Partial Autocorrelation:

Autocorrelation Function (ACF):

- The function: $\rho(k) = \frac{\gamma(k)}{\gamma(0)}$, where $\gamma(k) = \text{Cov}(X_t, X_{t-k})$.

Partial Autocorrelation Function (PACF):

- The correlation between X_t and X_{t-k} , controlling for all intermediate lags $X_{t-1}, \dots, X_{t-k+1}$.

Identification Rule:

Model	ACF Behavior	PACF Behavior
AR(p)	Decays gradually	Cuts off after lag p
MA(q)	Cuts off after lag q	Decays gradually
ARMA(p,q)	Decays gradually	Decays gradually

Task 6. (Plotting ACF and PACF)

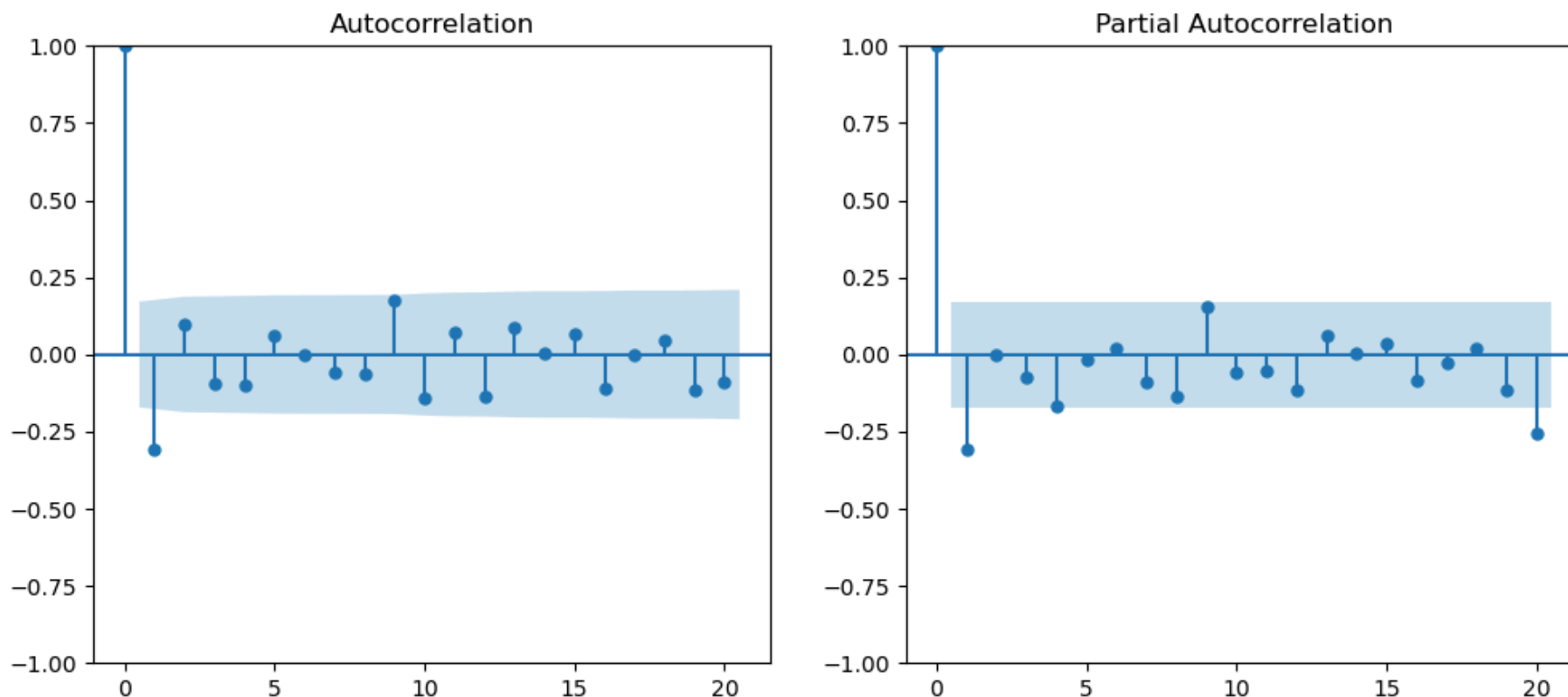
Once we have stationary data, we can use Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) to identify models.

Instructions:

1. Import the plotting functions from `statsmodels.graphics.tsaplots`.
2. Create side-by-side plots for 20 lags using `plt.subplots(1, 2, figsize=(12, 5))`.
3. Inspect the plots and decide whether the process resembles AR(p), MA(q), or ARMA(p, q).

```
In [48]: from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_acf(passengers_diff_1_12, lags=20, ax=axes[0])
plot_pacf(passengers_diff_1_12, lags=20, ax=axes[1])
plt.show()
```



Basing on the plots, we can see that both autocorrelation and partial autocorrelation have some observations (apart from the 0th one) not fitting in the blue confidence interval, so we should use ARMA to be able to capture both. The most efficient one seems to be the ARMA(1,1), because in both of them the first observation seems to be a good place to start treating the following observations as a noise.

Task 7. (Fitting ARMA models)

Now that we have a stationary series, we can try to fit some simple ARMA models. We will consider AR(1), MA(1), and ARMA(1,1).

Instructions:

1. Store our stationary data as a NumPy array: `y = passengers_diff_1_12.to_numpy().flatten()` .

2. Use `ARIMA` from `statsmodels.tsa.arima.model` .

- For AR(p), set `order=(p,0,0)`
- For MA(q), set `order=(0,0,q)`
- For ARMA(p,q), set `order=(p,0,q)`

3. Fit models to `y` using `.fit()` .

4. Print `.summary()` for each.

5. Compare the models based on:

- AIC - Akaike Information Criterion (lower is better)
- BIC - Bayesian Information Criterion (lower is better)
- Log-likelihood - (higher is better)
- p-value of parameters (look for < 0.05)

6. Check residuals:

- Plot residuals, ACF/PACF of residuals, and histogram.
- Use a 2x2 grid for plots.
- Residuals should look like white noise: no autocorrelation and symmetric around zero. It would be good if they were approximately normal.

```
In [8]: from statsmodels.tsa.arima.model import ARIMA
y = passengers_diff_1_12.to_numpy().flatten()

arima_ar=ARIMA(y, order=(1,0,0))
arima_ma=ARIMA(y, order=(0,0,1))
arima_arma=ARIMA(y, order=(1,0,1))

ar=arima_ar.fit()
ma=arima_ma.fit()
arma=arima_arma.fit()

print(f"AR MODEL\n{ar.summary()}\n\n")
print(f"MA MODEL\n{ma.summary()}\n\n")
print(f"ARMA MODEL\n{arma.summary()}")
```

AR MODEL

SARIMAX Results

```

=====
Dep. Variable:          y    No. Observations:          131
Model:                ARIMA(1, 0, 0)    Log Likelihood          -508.171
Date:                Fri, 09 Jan 2026    AIC                  1022.343
Time:                19:37:27    BIC                  1030.969
Sample:                0    HQIC                  1025.848
                        - 131
Covariance Type:          opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	0.1773	0.790	0.224	0.823	-1.372	1.726
ar.L1	-0.3078	0.074	-4.146	0.000	-0.453	-0.162
sigma2	136.9552	14.615	9.371	0.000	108.310	165.600

```

=====
Ljung-Box (L1) (Q):          0.00    Jarque-Bera (JB):          7.74
Prob(Q):                    0.98    Prob(JB):                0.02
Heteroskedasticity (H):      2.34    Skew:                    0.00
Prob(H) (two-sided):         0.01    Kurtosis:                4.19
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

MA MODEL

SARIMAX Results

```

=====
Dep. Variable:          y    No. Observations:          131
Model:                ARIMA(0, 0, 1)    Log Likelihood          -508.281
Date:                Fri, 09 Jan 2026    AIC                  1022.563
Time:                19:37:27    BIC                  1031.188
Sample:                0    HQIC                  1026.068
                        - 131
Covariance Type:          opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
--	------	---------	---	------	--------	--------

```

const          0.1938      0.698      0.278      0.781      -1.174      1.562
ma.L1          -0.3196      0.063     -5.048      0.000      -0.444      -0.196
sigma2         137.1843     15.012      9.139      0.000     107.762     166.606
=====
Ljung-Box (L1) (Q):          0.00   Jarque-Bera (JB):          5.41
Prob(Q):          0.96   Prob(JB):          0.07
Heteroskedasticity (H):      2.38   Skew:          0.02
Prob(H) (two-sided):        0.00   Kurtosis:         3.99
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

ARMA MODEL

SARIMAX Results

```

=====
Dep. Variable:          y   No. Observations:          131
Model:                ARIMA(1, 0, 1)   Log Likelihood          -508.171
Date:                Fri, 09 Jan 2026   AIC                  1024.342
Time:                19:37:27   BIC                  1035.843
Sample:                0   HQIC                  1029.016
                        - 131
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.1771      0.791      0.224      0.823      -1.373      1.727
ar.L1         -0.2989      0.359     -0.832      0.406      -1.003      0.405
ma.L1         -0.0099      0.338     -0.029      0.977      -0.673      0.653
sigma2        136.9617     14.646      9.352      0.000     108.257     165.667
=====
Ljung-Box (L1) (Q):          0.00   Jarque-Bera (JB):          7.66
Prob(Q):          0.98   Prob(JB):          0.02
Heteroskedasticity (H):      2.34   Skew:          0.00
Prob(H) (two-sided):        0.01   Kurtosis:         4.18
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
In [64]: mod_names=["AR", "MA", "ARMA"]
comparison={
    "model": mod_names,
    "aic": [ar.aic, ma.aic, arma.aic],
    "bic": [ar.bic, ma.bic, arma.bic],
    "loglikelihood": [ar.llf, ma.llf, arma.llf]
}

print(f"AIC, BIC and loglikelihood comparison:\n\n{pd.DataFrame(comparison)}\n")
print(f"P-values:\n{pd.DataFrame([ar.pvalues, ma.pvalues, arma.pvalues], index=mod_names)}")
```

AIC, BIC and loglikelihood comparison:

	model	aic	bic	loglikelihood
0	AR	1022.342991	1030.968583	-508.171496
1	MA	1022.562788	1031.188380	-508.281394
2	ARMA	1024.342444	1035.843233	-508.171222

P-values:

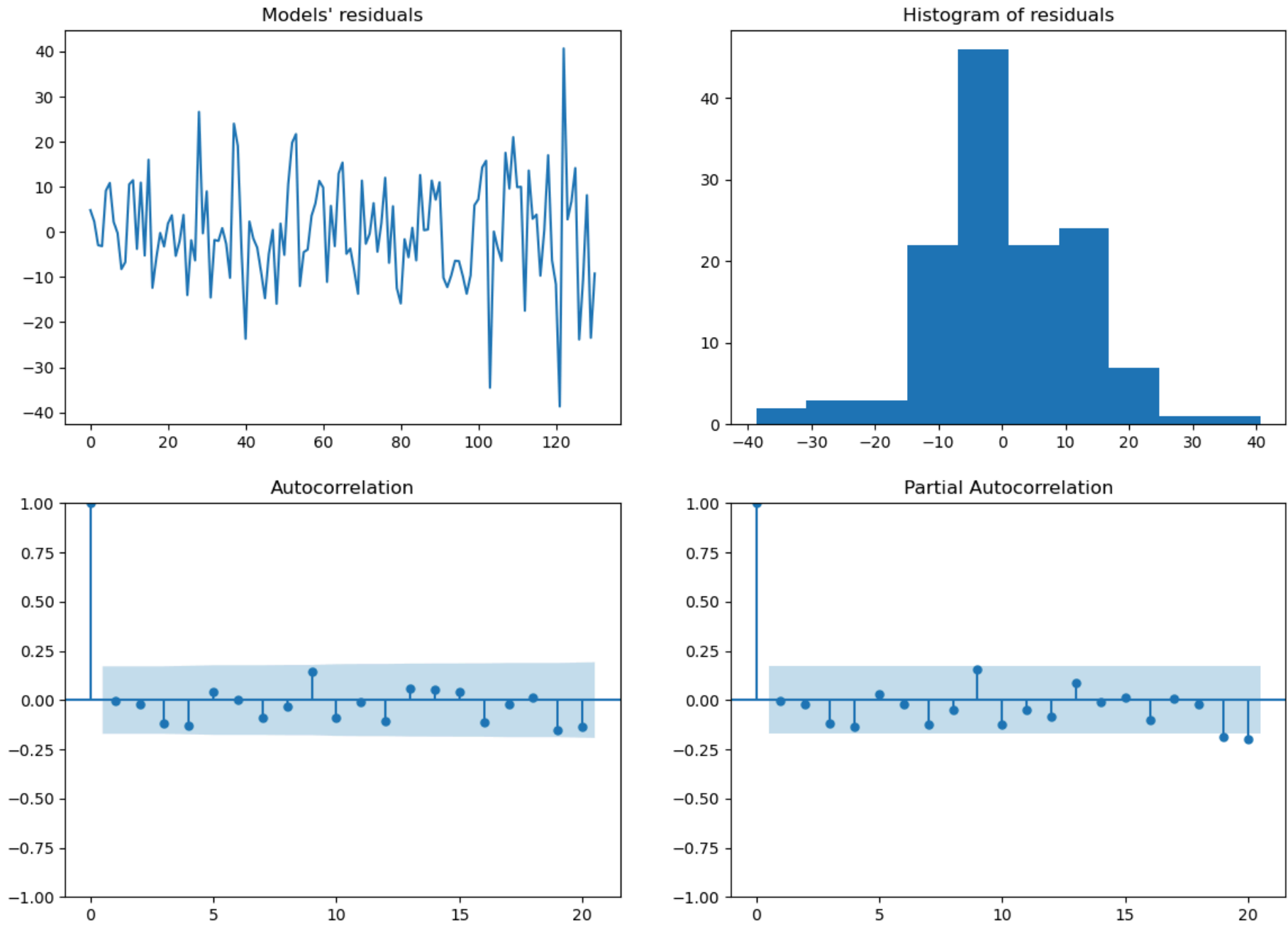
	0	1	2	3
AR	0.822508	3.382901e-05	7.191084e-21	NaN
MA	0.781272	4.461914e-07	6.327728e-20	NaN
ARMA	0.822735	4.055459e-01	9.766923e-01	8.630354e-21

AIC and BIC should be as low as possible, while loglikelihood as big as possible, so model AR seems to be the best. Its p-values (apart from the constant) have p-value < 0.05 , so we can agree that AR is the best of them.

```
In [55]: def residual_plotting(model):
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes[0,0].plot(model.resid)
axes[0,0].set_title("Models' residuals")
axes[0,1].hist(model.resid)
axes[0,1].set_title("Histogram of residuals")
plot_acf(model.resid, lags=20, ax=axes[1,0])
plot_pacf(model.resid, lags=20, ax=axes[1,1])
plt.show()
```

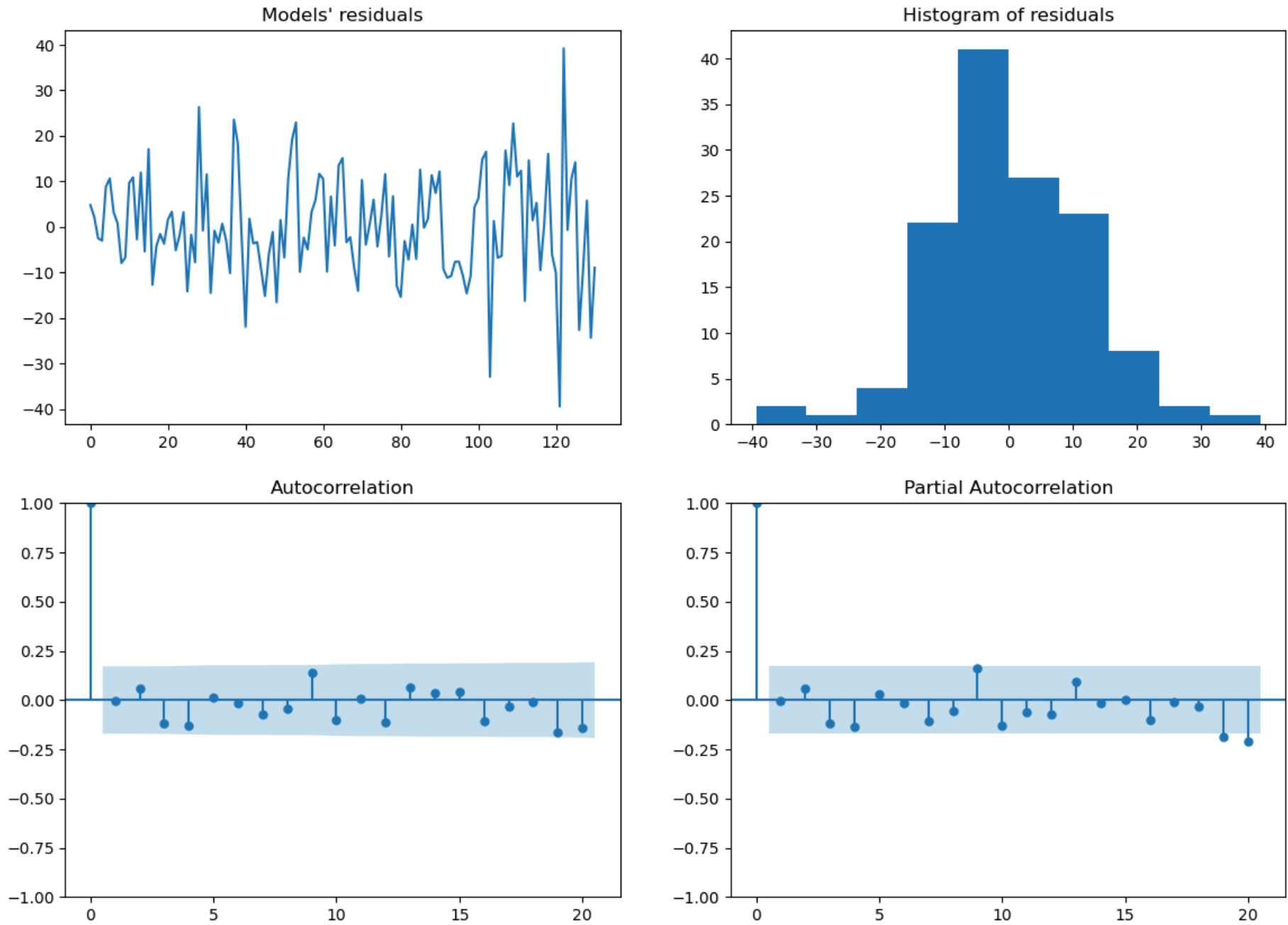
MODEL AR

```
In [56]: residual_plotting(ar)
```

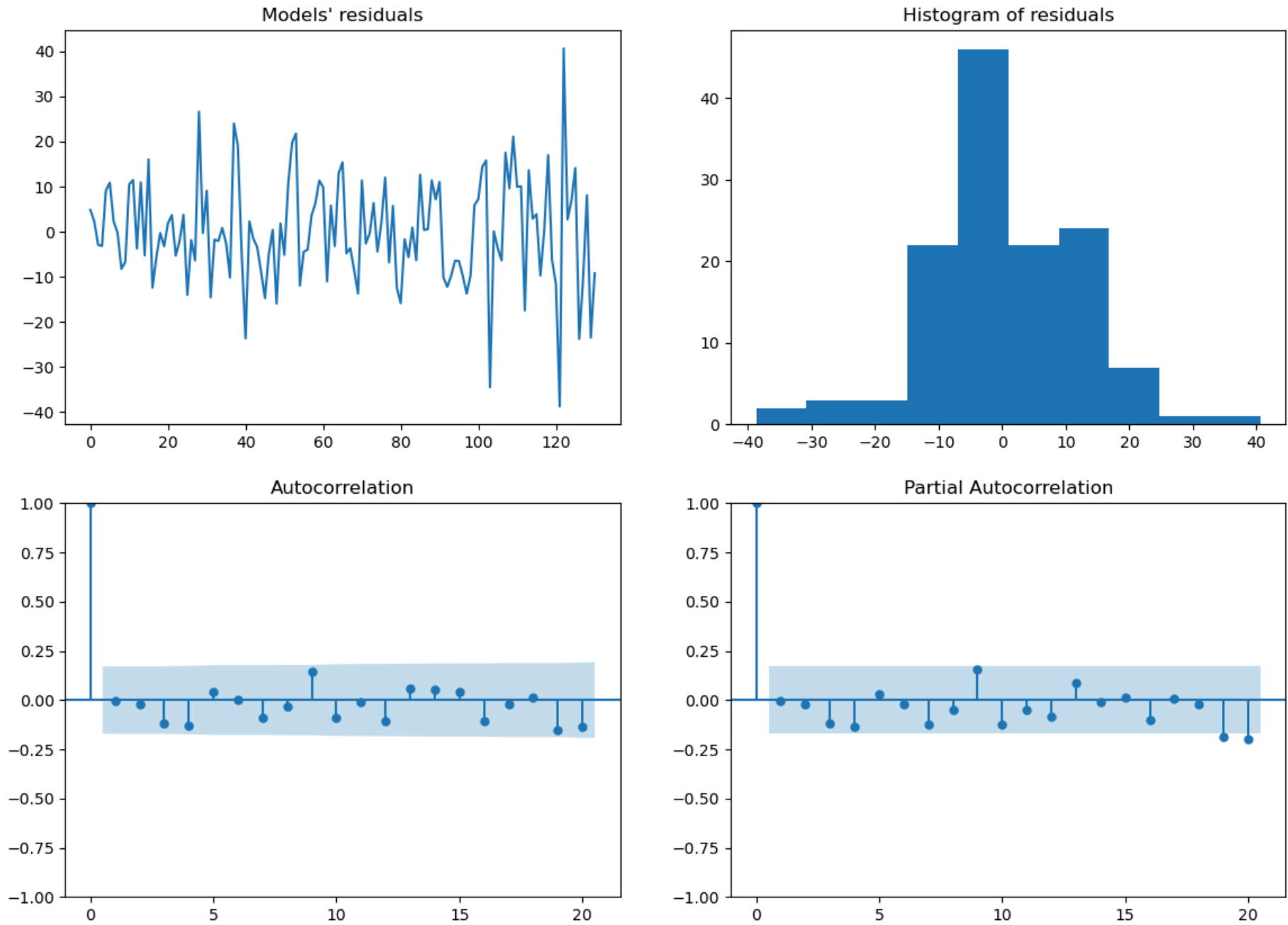
MODEL MA

```
In [43]: residual_plotting(ma)
```



MODEL ARMA

```
In [44]: residual_plotting(arma)
```



We can see that all of our models seem to be correct - residuals look like white noise: no autocorrelation (apart from 0th one and some partial in the end), fairly symmetric around zero and approximately normal. We're still going to choose AR one, because of the aforementioned best results in a summary.

Task 8. (Forecasting with ARMA and Inverse Differencing)

After differencing, forecasts from ARMA models are predictions of the differences:

To get back to the original scale, we must invert differencing step by step:

1. Fit an ARMA model using `ARIMA(y, order=(p,0,q))` and `.fit()`.
2. Forecast the differenced series with `.get_forecast(steps=7).predicted_mean`.
3. Convert the training series to NumPy:

```
Y_new = passengers_train.to_numpy().flatten().copy()
```

We need this because the recursion formula uses the last 13 original values.

4. For each forecasted difference, compute:

$$\hat{Y}_t = Y_{t-1} + Y_{t-12} - Y_{t-13} + \hat{Y}_t''$$

Append each forecast to `Y_new`.

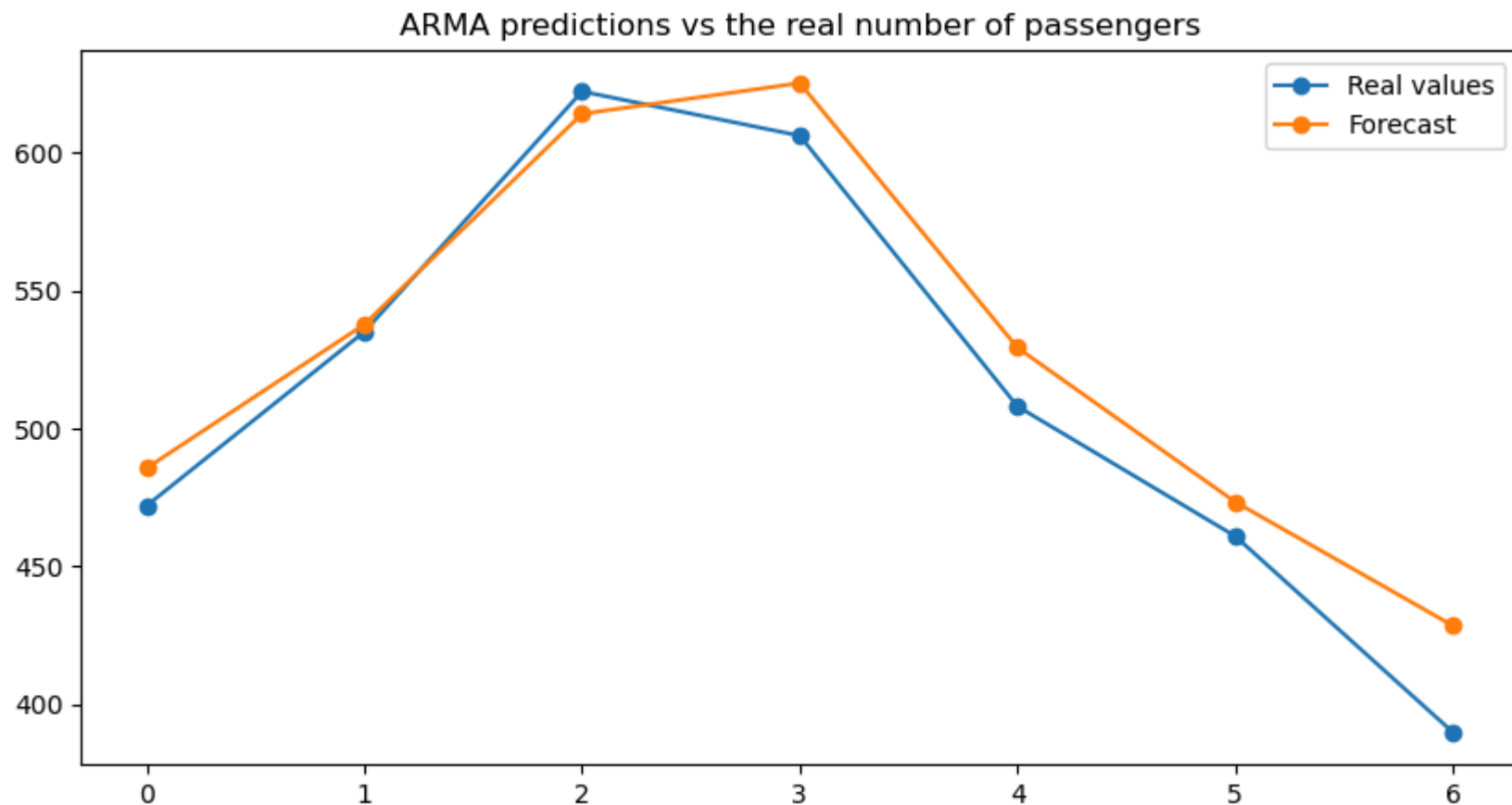
5. Extract the last 7 values from `Y_new` as forecasts in the original scale.
6. Compare with the first 7 values of `passengers_test`. Plot both on the same figure with labels, legend, and title.

```
In [72]: #1) - we already have arma
arma_diff=arma.get_forecast(steps=7).predicted_mean
Y_new = passengers_train.to_numpy().flatten().copy()

for diff in arma_diff:
    Y_t=Y_new[-1]+Y_new[-12]-Y_new[-13]+diff #Y_new[-1]=Y_(t-1) and so on
    Y_new=np.append(Y_new, Y_t)
```

```
forecast=Y_new[-7:]
real_val=passengers_test[:7].to_numpy().flatten().copy()

plt.figure(figsize=(10,5))
plt.plot(real_val, label="Real values", marker="o")
plt.plot(forecast, label="Forecast", marker="o")
plt.title("ARMA predictions vs the real number of passengers")
plt.legend()
plt.show()
```



Alternative Method to Remove the Trend

So far, we removed the trend using differencing. Now, let us consider an alternative approach: estimating the trend directly using OLS regression.

Task 9. (NOT MANDATORY) (Logarithmic Transformation)

So far, we removed the trend using differencing. Now let us try an alternative method: estimating the trend directly with OLS regression.

Because variance increases over time, we first apply a logarithmic transformation:

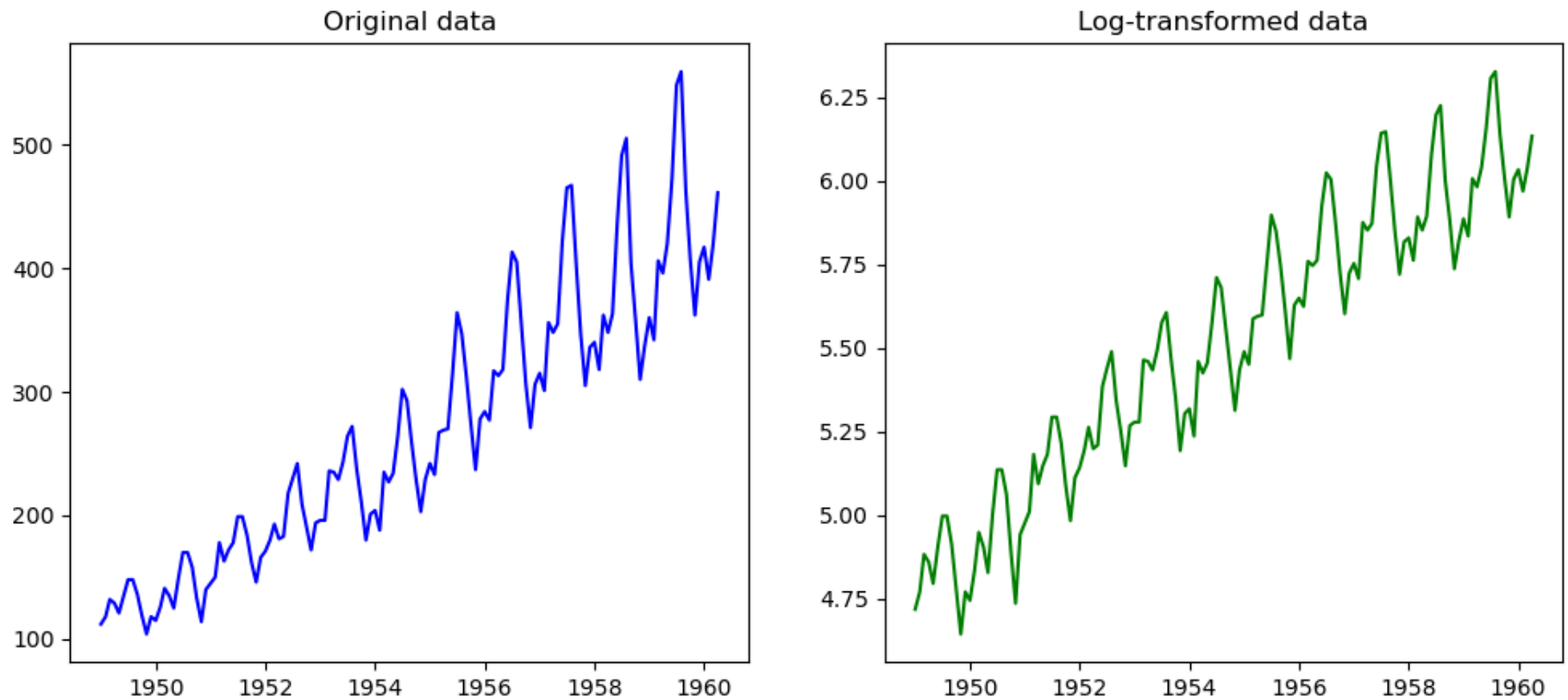
$$U_t = \log(Y_t)$$

Instructions:

1. Apply `np.log()` to the series and store in `passengers_log`.
2. Plot both the original and the log-transformed series in separate figures.

```
In [52]: passengers_log=np.log(passengers_train)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].plot(passengers_train, color="blue")
axes[0].set_title("Original data")
axes[1].plot(passengers_log, color="green")
axes[1].set_title("Log-transformed data")
plt.show()
```

Task 10. (NOT MANDATORY) (Estimating the Trend using OLS Regression)

We can estimate the trend of $U_t = \log(Y_t)$ using Ordinary Least Squares (OLS) regression. Two common specifications are:

- Linear trend:

$$V_t = \beta_0 + \beta_1 t$$

- Quadratic trend:

$$V_t = \beta_0 + \beta_1 t + \beta_2 t^2$$

Instructions:

1. Create a time index `t = 0, 1, 2, ....`
2. Fit two regression models using `statsmodels` : linear and quadratic:
 - use `sm.OLS(y, X)`
 - `y` is the dependent variable (our data, here `log(Y_t)`),
 - `X` is the design matrix of regressors (e.g. for quadratic trend `t` , `t^2`).
 - use `X = sm.add_constant(X)` . This ensures the model estimates both slope(s) and intercept.
3. Compare summaries (coefficients, R^2 , p-values).
4. Plot fitted trend lines against the log-transformed data.
5. Compute residuals (original minus fitted) as the detrended series.

In []: